

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ingeniería Mecatrónica, Electrónica y Biomédica
IE3054: Electrónica Digital 2

PROYECTO NO. 3

CONTROL DE REGISTRO PARA PARQUEO

Juan Antonio Rodríguez Pineda - 221593
Gerardo López -

Guatemala, mayo 2026

1. Descripción del Proyecto

El presente proyecto corresponde al Proyecto No. 3 del curso IE3054 – Electrónica Digital 2, cuyo objetivo es diseñar e implementar un sistema embebido de control de registro para un parqueo. El sistema funciona como plan piloto para la empresa Parqueo-matic, con la finalidad de recopilar en tiempo real la ocupación de espacios disponibles en un nivel de sótano con 2 grupos de 4 parqueos (8 en total).

El sistema envía notificaciones al servidor de la empresa cada vez que existe un cambio de estado en alguno de los parqueos. Para ello se integran múltiples microcontroladores (Núcleo STM32F446RE y ESP32), protocolos de comunicación (I2C, UART), sensores de efecto Hall de bajo costo, Neopixels como indicadores visuales, una pantalla TFT con interfaz gráfica y un servidor web accesible vía WiFi.

2. Objetivos

2.1 Objetivo General

Diseñar e implementar un sistema embebido que permita monitorear en tiempo real la ocupación de un parqueo de 8 plazas, enviando notificaciones a un servidor web y mostrando el estado visual mediante indicadores físicos (Neopixels) y una interfaz gráfica (TFT).

2.2 Objetivos Específicos

- Implementar sensores de efecto Hall (A3141) de bajo costo para detectar la presencia de vehículos en cada plaza mediante interrupciones GPIO (EXTI).
- Controlar 8 Neopixels WS2812B como indicadores bicolores (verde = libre, rojo = ocupado) utilizando TIM1 con DMA para generar la señal de temporización precisa.
- Desarrollar una interfaz gráfica (GUI) web tipo Mario Bros en el ESP32, mostrando el estado de todos los parqueos con animaciones CSS y actualización en tiempo real.
- Establecer comunicación I2C entre la Núcleo STM32F446RE (esclava, dirección 0x18) y el ESP32 (maestro), con mecanismos de recuperación robusta ante fallos del bus.
- Configurar el ESP32 como Access Point WiFi (SSID: Parqueo_ESP32) y servidor web HTTP en el puerto 80 para transmitir el estado del parqueo.
- Implementar un sistema de alimentación portátil basado en batería LiPo, módulo TP4056, switch de encendido y regulador de voltaje Blackout 5 V.
- Implementar un Watchdog de hardware (IWDG) y estrategias de recuperación de software para garantizar la robustez de la comunicación I2C.

3. Materiales Utilizados

- Núcleo STM32F446RE (MCU principal, modo esclavo I2C)
- ESP32 (Access Point WiFi, maestro I2C, servidor web)
- Pantalla TFT ILI9341 320×240 px (interfaz gráfica GUI)
- Protoboard
- 8× Neopixels WS2812B (indicadores bicolores por espacio)
- 8× Sensores de efecto Hall Allegro A3141 (detección de vehículos mediante imán permanente)
- 1× Display de 7 segmentos o LCD (indicador de espacios libres)
- Resistencias variadas (pull-up I2C, limitadoras de corriente)
- Batería LiPo 3.7 V
- Módulo TP4056 (cargador/protector de batería LiPo)
- Switch de encendido/apagado
- Regulador de voltaje Blackout 5 V

4. Sensor de Efecto Hall A3141

Para la detección de vehículos se utilizaron sensores de efecto Hall Allegro A3141. Estos dispositivos detectan la presencia de un campo magnético unipolar: cuando un imán permanente se acerca al sensor (simulando la presencia de un vehículo con un imán adherido), la salida de colector abierto conmuta a LOW (activa), y cuando el imán se aleja la salida vuelve a HIGH (inactiva gracias al pull-up externo).

4.1 Principio de Operación en el Proyecto

Cada sensor A3141 se conectó a un pin GPIO de la Núcleo configurado en modo EXTI (Rising + Falling edge). Un imán permanente se coloca bajo cada espacio de parqueo. Cuando un vehículo está ausente, el imán está cerca del sensor → salida LOW → interrupción FALLING → bit de parqueo = OCUPADO. Cuando el vehículo está presente (cubre el imán alejándolo del sensor) → salida HIGH → interrupción RISING → bit de parqueo = LIBRE.

Este diseño de bajo costo es ideal para implementar en los 10 sótanos del edificio, ya que los sensores Hall A3141 son económicos y no requieren calibración compleja como los sensores ultrasónicos. Aparte permite realizar una instalación oculta del sensor, sin afectar el funcionamiento.

5. Arquitectura del Sistema

5.1 Diagrama de Conexión

El sistema sigue la arquitectura propuesta en la guía del proyecto. Los bloques principales son:

- Núcleo STM32F446RE: Lee 4 sensores Hall por interrupciones EXTI en pines PC2, PC3, PC10, PC12. Controla 4 Neopixels WS2812B por TIM1 CH1 con DMA2 Stream1. Se comunica como esclavo I2C1 (PB8=SCL, PB9=SDA, dirección 0x18). Depura por USART2 a 115200 bps.
- ESP32: Maestro I2C (GPIO21=SDA, GPIO22=SCL). Access Point WiFi SSID 'Parqueo_ESP32', contraseña '12345678'. Servidor web HTTP en puerto 80 con rutas '/' y '/status'.
- Pantalla TFT ILI9341: Conectada al Núcleo2 por SPI, muestra la GUI gráfica del estado del parqueo.
- Núcleo STM32F446RE 2: Este se encarga de monitorear los 4 sensores restantes, controlar los neopixeles WS2812B, es un esclavo con la dirección 0x27. Esta núcleo es la encargada de la muestra gráfica por lo que aparte de transmitir la información de los sensores por I2C a la ESP32 este también recibe los otros sensores de la núcleo.

5.2 Sistema de Alimentación Portátil

Para dotar al sistema de portabilidad se implementó una cadena de alimentación compuesta por:

- Batería LiPo 3.7 V: fuente principal de energía.
- Módulo TP4056: cargador/protector LiPo con protección contra sobrecarga, sobredescarga y cortocircuito.
- Switch SPST: controla el encendido/apagado de todo el sistema.
- Regulador Blackout 5 V: convierte la tensión variable de la batería a 5 V estables para los Neopixels, la Núcleo y el ESP32.

5.3 Protocolo de Comunicación I2C

El ESP32 actúa como maestro del bus I2C. El flujo de comunicación en cada ciclo de 500 ms es:

- El ESP32 abre transmisión a la dirección esclava 0x18 y envía el byte 'S' (ASCII 83) como señal de solicitud.
- La Núcleo recibe el byte en HAL_I2C_SlaveRxCpltCallback, actualiza el timestamp de actividad y limpia el buffer.
- El ESP32 hace requestFrom(0x18, 1) para leer 1 byte de respuesta.
- La Núcleo responde en HAL_I2C_SlaveTxCpltCallback enviando flagSensor_ESP32, que contiene los 4 bits de estado de los parqueos (1=ocupado, 0=libre).
- El ESP32 interpreta los bits y actualiza el arreglo 'espacios[8]' para la página web.
- Se repite el proceso para la segundo núcleo.
- La ESP32 le manda a la núcleo 2 el estado de los otros sensores HAL, para que esta pueda acceder a ellos y actualizar la muestra gráfica.

6. Código Comentado – Núcleo STM32F446RE

A continuación se presenta el código completo del archivo main.c de la Núcleo STM32, dividido en bloques funcionales con sus explicaciones correspondientes.

6.1 Includes, Defines y Variables Globales

Se incluye la librería NeoPixel.h (biblioteca propia para control de WS2812B por PWM+DMA) y se definen los tamaños de buffer I2C. Las variables más importantes son:

- flagSensor (uint8_t): byte de 8 bits donde los bits 0-3 (nibble bajo) indican OCUPADO y los bits 4-7 (nibble alto) indican LIBRE para cada parqueo. Se actualiza desde la ISR de EXTI.
- flagSensor_ESP32 (uint8_t): versión limpia de flagSensor lista para enviar al ESP32 (1=ocupado, 0=libre por bit).
- RGB1–RGB4 (uint8_t[3]): arreglos de color {R, G, B} para cada uno de los 4 Neopixels.
- i2c_last_activity: timestamp en ms del último evento I2C (usado por el watchdog de software).
- i2c_error_flag: bandera que el callback de error I2C levanta para que el main muestre retroalimentación visual.

```
#include "main.h"
#include "NeoPixel.h"    // Librería WS2812B por TIM1+DMA
#include <string.h>

#define TXbuffersize 4    // Tamaño buffer de transmisión I2C
#define RXbuffersize 4    // Tamaño buffer de recepción I2C

// Handles de periféricos generados por STM32CubeMX
I2C_HandleTypeDef hi2c1;
IWDG_HandleTypeDef hiwdg;
TIM_HandleTypeDef htim1;
DMA_HandleTypeDef hdma_tim1_ch1;
UART_HandleTypeDef huart2;

// Variables de Neopixeles (un arreglo RGB por cada LED)
volatile uint8_t flagSensor = 0; // Estado crudo de sensores (ISR)
volatile uint8_t flagSensor_ESP32 = 0; // Estado listo para I2C
uint8_t RGB1[3], RGB2[3], RGB3[3], RGB4[3];

// Variables I2C
uint8_t aTxBuffer[TXbuffersize]; // Buffer de transmisión al ESP32
uint8_t aRxBuffer[RXbuffersize]; // Buffer de recepción del ESP32

// Watchdog de software para I2C
volatile uint32_t i2c_last_activity = 0; // Tick del último evento I2C
volatile uint8_t i2c_error_flag = 0; // Bandera de error grave
#define I2C_TIMEOUT_MS 5000 // Umbral de timeout: 5 s
```

6.2 Funciones de Control de Neopixels

Se implementaron tres funciones para manejar los Neopixels de forma modular:

- `update_neopixels()`: limpia el buffer de Neopixel y envía los 4 colores actuales al bus WS2812B mediante DMA.
- `update_colors()`: recibe el byte de estado y una máscara, y determina si un parqueo específico está ocupado (rojo) o libre (verde), actualizando el arreglo RGB correspondiente.
- `update_park()`: llama a `update_colors()` para los 4 parqueos con sus respectivas máscaras de bit.

```
// Envía el estado actual de los 4 LEDs al bus WS2812B
void update_neopixels() {
    pixelClear(); // Limpiar buffer interno
    setPixelColor(0, RGB1[0], RGB1[1], RGB1[2]); // Parqueo 1
    setPixelColor(1, RGB2[0], RGB2[1], RGB2[2]); // Parqueo 2
    setPixelColor(2, RGB3[0], RGB3[1], RGB3[2]); // Parqueo 3
    setPixelColor(3, RGB4[0], RGB4[1], RGB4[2]); // Parqueo 4
    setBrightness(100); // Brillo al 100%
    pixelShow(); // Enviar por DMA (TIM1 CH1)
}

// Actualiza el color de un LED según el nibble de flagSensor
// compare = máscara del bit bajo (ej: 0x01 para parqueo 0)
// nibble alto (compare<<4) indica que el sensor volvió a LIBRE
void update_colors(uint8_t flagSensor, uint8_t RGB[3], uint8_t compare) {
    if ((flagSensor & compare) == compare) {
        RGB[0] = 255; RGB[1] = 0; // ROJO → parqueo OCUPADO
    } else if ((flagSensor & (compare<<4)) == (compare<<4)) {
        RGB[0] = 0; RGB[1] = 255; // VERDE → parqueo LIBRE
    }
}

// Actualiza los 4 parqueos con sus máscaras correspondientes
void update_park(uint8_t flagSensor) {
    update_colors(flagSensor, RGB1, 0x01); // PC10 → parqueo 0
    update_colors(flagSensor, RGB2, 0x02); // PC2 → parqueo 1
    update_colors(flagSensor, RGB3, 0x04); // PC12 → parqueo 2
    update_colors(flagSensor, RGB4, 0x08); // PC3 → parqueo 3
}
```

6.3 Función main() – Inicialización y Loop Principal

El main inicializa todos los periféricos, detecta si hubo un reset por IWDG, arranca en verde (todos libres) y entra al loop. El loop principal tiene 4 responsabilidades:

- Alimentar el IWDG periódicamente para evitar reset por watchdog de hardware.
- Verificar `i2c_error_flag` y mostrar retroalimentación visual (parpadeo azul) si hubo un error I2C grave que ya fue recuperado.
- Llamar a `update_park()` y `update_neopixels()` si `flagSensor` cambió.

- Verificar el watchdog de software: si no hubo actividad I2C en 5 s, reiniciar el periférico I2C.

```
int main(void) {
    HAL_Init();
    SystemClock_Config(); // Clock a 84 MHz (HSI + PLL)
    MX_GPIO_Init();       // GPIOs: sensores (EXTI) + LED LD2
    MX_DMA_Init();         // DMA2 Stream1 para TIM1 CH1 (Neopixel)
    MX_USART2_UART_Init(); // UART debug a 115200 bps
    MX_I2C1_Init();         // I2C1 esclavo, dir=0x18, 100 kHz
    MX_TIM1_Init();         // TIM1 PWM period=104 para WS2812B
    MX_IWDG_Init();         // IWDG prescaler=64, reload=4095 (~26 s)

    // Detectar reset por IWDG y notificar por UART
    if (__HAL_RCC_GET_FLAG(RCC_FLAG_IWDGRST)) {
        HAL_RCC_CLEAR_RESET_FLAGS();
        HAL_UART_Transmit(&huart2,
            (uint8_t*)"Reset por IWDG - I2C recuperado\r\n", 33, 100);
    }

    // Estado inicial: todos los parqueos libres (verde)
    RGB1[1]=255; RGB2[1]=255; RGB3[1]=255; RGB4[1]=255;
    HAL_I2C_EnableListen_IT(&hi2c1); // Habilitar escucha I2C en modo IT
    update_neopixels();
    HAL_UART_Transmit(&huart2, (uint8_t*)"Estoy vivo\r\n", 12, 100);

    while (1) {
        HAL_IWDG_Refresh(&hiwdg); // Alimentar IWDG (evitar reset HW)

        // Retroalimentación visual de error I2C recuperado
        if (i2c_error_flag) {
            i2c_error_flag = 0;
            // Parpadeo azul: indica error pero recuperado
            RGB1[2]=255; RGB1[0]=0; RGB1[1]=0; // (y similar para RGB2-4)
            update_neopixels(); HAL_Delay(500);
            RGB1[2]=0; RGB1[1]=255; RGB1[0]=0;
            update_neopixels();
        }

        // Actualizar Neopixels solo si hubo cambio de sensor
        update_park(flagSensor);
        if (flagSensor != 0) { update_neopixels(); flagSensor = 0; }

        // Watchdog de software I2C: reiniciar si no hay actividad en 5 s
        if (i2c_last_activity != 0 &&
            (HAL_GetTick() - i2c_last_activity) > I2C_TIMEOUT_MS) {
            i2c_last_activity = HAL_GetTick();
            HAL_I2C_DeInit(&hi2c1);
            HAL_Delay(10);
            HAL_I2C_Init(&hi2c1);
            HAL_I2C_EnableListen_IT(&hi2c1);
        }
        HAL_Delay(10);
    }
}
```

6.4 Inicialización de I2C con Recuperación de Bus

Un problema común en I2C es que la línea SDA quede 'colgada' en LOW si una transacción fue interrumpida. Para solucionarlo, antes de inicializar el periférico I2C1, se ejecuta una recuperación manual del bus: se configuran PB8 (SCL) y PB9 (SDA) como GPIO open-drain y se generan hasta 9 pulsos de clock si SDA está en LOW, seguidos de una condición STOP manual.

```
// MX_I2C1_Init - PARTE 1: Recuperación manual del bus I2C
// (antes de que HAL_I2C_Init tome control de los pines)
GPIO_InitTypeDef GPIO_InitStruct = {0};
GPIO_InitStruct.Pin = GPIO_PIN_8 | GPIO_PIN_9;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD; // Open-drain (igual que I2C)
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET); // SCL = HIGH
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_SET); // SDA = HIGH

// Si SDA está en LOW → bus bloqueado → generar pulsos de SCL
if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_9) == GPIO_PIN_RESET) {
    for (int i = 0; i < 9; i++) {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET); // SCL LOW
        HAL_Delay(1);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET); // SCL HIGH
        HAL_Delay(1);
    }
}

// Generar condición STOP manual (SDA LOW → HIGH con SCL HIGH)
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_RESET); HAL_Delay(1);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_SET); HAL_Delay(1);

// MX_I2C1_Init - PARTE 2: Configuración normal del periférico
hi2c1.Instance = I2C1;
hi2c1.Init.ClockSpeed = 100000; // 100 kHz
hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
hi2c1.Init.OwnAddress1 = 48; // 0x18 << 1 = 48 (formato HAL)
hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
HAL_I2C_Init(&hi2c1);
```

6.5 IWDG – Watchdog de Hardware

El IWDG (Independent Watchdog) es un watchdog de hardware alimentado por el oscilador LSI (32 kHz), completamente independiente del clock principal. Si el programa se congela (por ejemplo, si la tarea I2C bloquea el loop), el IWDG no recibirá el Refresh y reseteará el microcontrolador automáticamente, recuperando el sistema.

```
// Configuración del IWDG
// Tiempo de timeout = (Reload + 1) × Prescaler / 32000
// = (4095+1) × 64 / 32000 ≈ 8.19 s
hiwdg.Instance = IWDG;
```



```

hiwdg.Init.Prescaler = IWDG_PRESCALER_64; // Divisor de 64
hiwdg.Init.Reload    = 4095;              // Valor máximo (12 bits)
HAL_IWDG_Init(&hiwdg);

// En el loop principal, se debe llamar periódicamente:
HAL_IWDG_Refresh(&hiwdg); // Resetea el contador del IWDG

// Al arrancar, verificar si el reset fue causado por IWDG:
if ( __HAL_RCC_GET_FLAG(RCC_FLAG_IWDGRST)) {
    __HAL_RCC_CLEAR_RESET_FLAGS();
    // → Notificar por UART que fue un reset de recuperación
}

```

6.6 Callbacks de Interrupción GPIO – Sensores Hall

Los 4 sensores Hall están conectados a pines PC2, PC3, PC10, PC12 configurados con EXTI Rising+Falling. Al detectar cambio de flanco, HAL_GPIO_EXTI_Callback actualiza flagSensor con un byte de 8 bits donde:

- Nibble bajo (bits 0-3): se pone en 1 cuando el sensor detecta LOW (imán presente → parqueo OCUPADO).
- Nibble alto (bits 4-7): se pone en 1 cuando el sensor vuelve a HIGH (imán ausente → parqueo LIBRE).

Finalmente, flagSensor_ESP32 combina ambos nibbles: OR del nibble bajo (fuerza 1 cuando ocupa) y AND-NOT del nibble alto desplazado (limpia el bit cuando se libera).

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    // PC10 → Parqueo 0 (bit 0)
    if (GPIO_Pin == GPIO_PIN_10) {
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_10) == GPIO_PIN_RESET)
            flagSensor |= 0x01; // Flanco FALLING → OCUPADO
        else
            flagSensor |= 0x10; // Flanco RISING → LIBRE
    }
    // PC2 → Parqueo 1 (bit 1)
    if (GPIO_Pin == GPIO_PIN_2) {
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_2) == GPIO_PIN_RESET)
            flagSensor |= 0x02;
        else
            flagSensor |= 0x20;
    }
    // PC12 → Parqueo 2 (bit 2)
    if (GPIO_Pin == GPIO_PIN_12) {
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_12) == GPIO_PIN_RESET)
            flagSensor |= 0x04;
        else
            flagSensor |= 0x40;
    }
    // PC3 → Parqueo 3 (bit 3)
    if (GPIO_Pin == GPIO_PIN_3) {
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_3) == GPIO_PIN_RESET)
            flagSensor |= 0x08;
        else
            flagSensor |= 0x80;
    }
}

```

```

}
// Actualizar flag para ESP32:
// OR con nibble bajo → marcar bits ocupados
// AND-NOT con nibble alto desplazado → limpiar bits liberados
flagSensor_ESP32 |= (flagSensor & 0x0F);
flagSensor_ESP32 &= ~(flagSensor >> 4);
}

```

6.7 Callbacks I2C – Comunicación con ESP32

La Nucleo opera como esclavo I2C en modo interrupción. Hay cuatro callbacks relevantes:

```

// Al completarse el ciclo de escucha → re-habilitar para el siguiente
void HAL_I2C_ListenCpltCallback(I2C_HandleTypeDef *hi2c) {
    HAL_I2C_EnableListen_IT(hi2c);
}

// Al detectar dirección I2C coincidente:
void HAL_I2C_AddrCallback(I2C_HandleTypeDef *hi2c,
                          uint8_t TransferDirection, uint16_t AddrMatchCode) {
    if (TransferDirection == I2C_DIRECTION_TRANSMIT) {
        // Maestro quiere ESCRIBIR → Nucleo recibe
        HAL_I2C_Slave_Seq_Receive_IT(&hi2c1, aRxBuffer, 4,
                                      I2C_FIRST_AND_LAST_FRAME);
    } else {
        // Maestro quiere LEER → Nucleo transmite flagSensor_ESP32
        HAL_I2C_Slave_Seq_Transmit_IT(&hi2c1, &aTxBuffer[0], 1,
                                      I2C_FIRST_AND_LAST_FRAME);
    }
}

// Recepción completada: ESP32 envió 'S' (ASCII 83)
void HAL_I2C_SlaveRxCpltCallback(I2C_HandleTypeDef *hi2c) {
    i2c_last_activity = HAL_GetTick(); // Actualizar timestamp watchdog
    if (aRxBuffer[0] == 83) {          // 'S' = solicitud de estado
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin); // LED debug
    }
    aRxBuffer[0] = 0x00;                // Limpiar buffer
}

// Transmisión completada: actualizar buffer con estado más reciente
void HAL_I2C_SlaveTxCpltCallback(I2C_HandleTypeDef *hi2c) {
    i2c_last_activity = HAL_GetTick();
    aTxBuffer[0] = flagSensor_ESP32; // Preparar para siguiente consulta
}

// Error I2C: manejo diferenciado según gravedad
void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c) {
    if (HAL_I2C_GetError(hi2c) == HAL_I2C_ERROR_AF) {
        // AF = Acknowledge Failure (normal) → solo re-escuchar
        HAL_I2C_EnableListen_IT(hi2c);
    } else {
        // Error grave → reiniciar periférico completamente
        HAL_I2C_DeInit(hi2c);
        HAL_I2C_Init(hi2c);
    }
}

```

```

    HAL_I2C_EnableListen_IT(hi2c);
    i2c_error_flag = 1; // Señalizar al main para retroalimentación
  }
}

```

7. Código Comentado – ESP32 (Maestro I2C + Servidor Web)

El ESP32 cumple tres roles simultáneos: maestro I2C que consulta el estado de los parqueos cada 500 ms, Access Point WiFi que crea la red 'Parqueo_ESP32', y servidor HTTP que sirve la página web y el endpoint JSON.

7.1 Definiciones y Variables Globales

```

#include <Wire.h>          // Biblioteca I2C de Arduino
#include <WiFi.h>          // Biblioteca WiFi del ESP32
#include <WebServer.h>     // Servidor HTTP

#define addressSlave1 0x18 // Dirección I2C de la Núcleo (grupo A)
#define addressSlave2 0x27 // Dirección I2C (grupo B, comentado)

const char* ssid      = "Parqueo_ESP32";
const char* password = "12345678";

WebServer server(80); // Servidor HTTP en puerto 80
bool espacios[8] = {0,0,0,0,0,0,0,0}; // Estado de los 8 parqueos
unsigned long ultimaActualizacion = 0;
const long intervalo = 500; // Consultar I2C cada 500 ms

#define SDA 21 // Pin SDA del ESP32
#define SCL 22 // Pin SCL del ESP32

```

7.2 Página Web HTML/CSS/JS (GUI Mario Bros)

La página web está almacenada en flash del ESP32 usando PROGMEM. Incluye:

- Estilo pixel-art con paleta de colores tipo Mario Bros (arena, bloques, tubos verdes).
- Grid de 4 columnas (o 2 en móvil) con 8 espacios de parqueo.
- Animación CSS de 'tubo Mario' para cada parqueo: cuando se ocupa, un tubo verde sube, sale un carro pixelado y el tubo baja.
- Sonido 8-bit mediante Web Audio API cuando cambia el estado de un parqueo.
- Botón 'Actualizar' que hace fetch('/status') y actualiza la vista sin recargar la página.
- Contador de espacios libres/ocupados en tiempo real.

```

// Fragmento clave del JavaScript en la página web:
function actualizar() {

```

```

const btn = document.getElementById('btnRefresh');
btn.classList.add('loading');
fetch('/status') // Llama al endpoint JSON
  .then(r => r.json())
  .then(d => renderGrid(d.espacios)) // Actualiza la vista
  .catch(() => location.reload()) // Fallback: recargar
  .finally(() => btn.classList.remove('loading'));
}

// Al cambiar el estado de un parqueo: reproducir tono Mario
function tonoMario() {
  const notas = [[523,0.10],[659,0.10],[784,0.10],[1047,0.20]];
  // ... genera los 4 tonos ascendentes con oscilador 'square'
}

```

7.3 Handlers del Servidor Web

```

// Genera el JSON de estado actual
String getEstadosJSON() {
  String json = "[";
  for (int i = 0; i < 8; i++) {
    json += (espacios[i] ? "true" : "false");
    if (i < 7) json += ",";
  }
  return json + "]";
}

// GET / → Sirve la página HTML con el estado actual incrustado
void handleRoot() {
  String html = HTML_PAGE;
  html.replace("%ESTADOS%", getEstadosJSON()); // Reemplazar placeholder
  server.send(200, "text/html", html);
}

// GET /status → Retorna JSON para polling AJAX
void handleStatus() {
  String json = "{\"espacios\":\"" + getEstadosJSON() + "\"}";
  server.send(200, "application/json", json);
}

```

7.4 Setup y Loop Principal del ESP32

```

void setup() {
  Serial.begin(115200);

  // Configurar como Access Point WiFi
  WiFi.softAP(ssid, password);
  IPAddress ip = WiFi.softAPIP();
  Serial.print("AP IP: "); Serial.println(ip); // → 192.168.4.1

  // Registrar rutas del servidor
  server.on("/", handleRoot); // Página principal HTML
  server.on("/status", handleStatus); // Endpoint JSON
}

```

```

server.begin();

// Inicializar I2C como maestro
Wire.begin(SDA, SCL);
i2cScanner(); // Escanear bus al arrancar (debug)
}

void loop() {
  server.handleClient(); // Atender peticiones HTTP

  if (millis() - ultimaActualizacion >= intervalo) { // Cada 500 ms
    ultimaActualizacion = millis();

    // 1. Enviar 'S' a la Nucleo (señal de solicitud de estado)
    Wire.beginTransaction(addressSlave1);
    Wire.write('S'); // ASCII 83
    error = Wire.endTransmission(true);

    // 2. Leer 1 byte de respuesta (flagSensor_ESP32)
    uint8_t estado = 0;
    if (Wire.requestFrom(addressSlave1, 1) > 0) {
      estado = Wire.read();
    }

    // 3. Interpretar cada bit como estado de parqueo
    for (int i = 0; i < 8; i++)
      espacios[i] = (estado & (0x01 << i)); // 1=ocupado, 0=libre
  }
}

```

8. Resultados

Se implementó exitosamente el sistema de control de registro para parqueo con los siguientes resultados:

- Los sensores Hall A3141 detectaron correctamente la presencia y ausencia del imán permanente mediante interrupciones EXTI, con respuesta inmediata y sin rebotes gracias a la histéresis interna del sensor.
- Los 4 Neopixels WS2812B cambiaron de color (verde ↔ rojo) en tiempo real mediante TIM1 PWM + DMA2, sin bloquear el loop principal.
- La comunicación I2C entre el ESP32 (maestro) y la Nucleo (esclava, 0x18) funcionó establemente. El sistema de recuperación en tres niveles (HAL_I2C_ErrorCallback, watchdog de software y IWDG) garantizó la robustez ante cortes de comunicación.
- El servidor web del ESP32 fue accesible desde cualquier dispositivo conectado a la red 'Parqueo_ESP32' (192.168.4.1), mostrando la GUI tipo Mario Bros con el estado en tiempo real y sonido 8-bit al cambiar el estado de un parqueo.
- El sistema de alimentación portátil (LiPo + TP4056 + Blackout 5V) proporcionó energía estable y el switch permitió encendido/apagado sin desconectar físicamente la batería.

Se identificaron áreas de mejora: la integración del segundo esclavo I2C (Núcleo 2, dirección 0x27) estaba implementada en el código pero comentada durante las pruebas piloto. La pantalla TFT GUI se realizó como servidor web en el ESP32 en lugar de pantalla física, simplificando el hardware final.

9. Rúbrica de Evaluación

Rubro	Punteo (%)
Implementación de sensores	10
Implementación de indicadores	15
Implementación de Interfaz GUI	15
Comunicación I2C con ESP32 y Núcleos	15
Web Server con el ESP32	25
Entregables semanales	10
Entregables	10
Total	100

10. Repositorio y Entregables

10.1 Repositorio GitHub

El código fuente completo del proyecto (STM32 y ESP32) está disponible en:
https://github.com/Juan-Rodriguez3/Digital_2/tree/main/ModuloSTM32/Nucleo1Paqueo

10.2 Video de Funcionamiento

El video demostrativo del funcionamiento del sistema se encuentra en YouTube:
<https://youtu.be/OT9wu0XWZl0>